Revisão 1: Consultas Básicas no Mysql

fantini@coltec.ufmg

21 de agosto de 2026

Sumário

1 Apresentação **2**

2 Revisão Consultas Básicas **3**

2.1 Exercício de Fixação 6

2.2 Tabela Produtos 8

3 Revisitando Operadores Lógicos **10**

3.1 Sintaxe dos Operadores AND, OR, NOT 10

4 Exercícios de Fixação **11**

5 Valores NULL **13**

5.1 Como testar valores NULL 13

6 Funções MAX() e MIN() **15**

7 Funções COUNT(), AVG(), SUM() **16**

8 Group By **16**

8.1 Exemplos com GROUP BY 17

8.2 Group By sem Funções Agregadoras 19

8.3 Exercícios de Fixação 20

9 Soluções **21**

9.1 Exercício de Fixação 26

10 Usando a Cláusula *Having* **26**

1 Apresentação

Nesse texto apresentamos uma revisão dos conceitos básicos de consultas em uma única tabela no Mysql.

São resumidos os conceitos apresentados na nossa apostila online.

Apresentamos exemplos de consultas realizadas e exercícios, com solções. Não olhe as soluções antes de fazer as consultas pedidas.

São 32 consultas com exemplos do USO de SELECT com as diversas opções, incluindo funções de agregação, operadores e funções.

São 5 listas de "Exercícios de Fixação", todos com soluções.

Usamos tabelas criadas pelo usuário e tabelas do banco de dados *northwind2024*.

2 Revisão Consultas Básicas

Select com Distinct

A instrução SELECT DISTINCT é usada para retornar apenas valores distintos (diferentes).

A tabela 1 apresenta dados de clientes registrados em uma empresa. Veja as duas consultas que informam os estados registrados na tabela.

Tabela 1: Tabela com Informações dos Clientes

ID	Nome	Endereco	Cidade	CEP	Estado
1	Jose Maria	Rua 1, num 2	Betim	12209	MG
2	Maria Jose	Rua 2, num 33	Maceio	5021	AL
3	Ana Maria	Rua A, num 4	Aracaju	5023	SE
4	Joana Carla	Rua 6, 7	BH	123	MG
5	João Carlos	Av. Flor	BH	12345	MG

```
mysql> select distinct Estado from Clientes;
+--------+
| Estado |
+--------+
| MG     |
| AL     |
| SE     |
+--------+
```

```
mysql> select Estado from Clientes;
+--------+
| Estado |
+--------+
| MG     |
| AL     |
| SE     |
| MG     |
| MG     |
+--------+
```

Usando o COUNT com Distinct

Podemos usar a palavra-chave DISTINCT para contar quantos estados temos registrados na tabela. Veja a diferença da consulta com e sem DISTINCT.

```
mysql> select count(distinct Estado) from Clientes;
+------------------------+
| count(distinct Estado) |
+------------------------+
|                      3 |
```

```
+----------------------+

mysql> select count(Estado) from Clientes;
+---------------+
| count(Estado) |
+---------------+
|             5 |
+---------------+
```

SELECT com WHERE

A cláusula WHERE é usada para filtrar registros.

É usado para extrair apenas os registros que atendem a uma condição especificada.

Selecione os clientes do Estado de Minas Gerais. Mostre todos os campos.

```
mysql> select * from Clientes where Estado = 'MG';
+------+--------------+-------------+--------+-------+--------+
| ID   | Nome         | Endereco    | Cidade | CEP   | Estado |
+------+--------------+-------------+--------+-------+--------+
|    1 | Jose Maria   | Rua 1, num 2 | Betim  | 12209 | MG     |
|    4 | Joana Carla  | Rua 6, 7     | BH     |   123 | MG     |
|    5 | João Carlos  | Av. Flor     | BH     | 12345 | MG     |
+------+--------------+-------------+--------+-------+--------+
```

Podemos usar a palavra-chave **LIKE**.

```
mysql> select * from Clientes where Estado like 'MG';
+------+--------------+-------------+--------+-------+--------+
| ID   | Nome         | Endereco    | Cidade | CEP   | Estado |
+------+--------------+-------------+--------+-------+--------+
|    1 | Jose Maria   | Rua 1, num 2 | Betim  | 12209 | MG     |
|    4 | Joana Carla  | Rua 6, 7     | BH     |   123 | MG     |
|    5 | João Carlos  | Av. Flor     | BH     | 12345 | MG     |
+------+--------------+-------------+--------+-------+--------+
3 rows in set (0.00 sec)
```

Obviamente podemos fazer consultas com condições para números também.

```
mysql> select * from Clientes where ID = 1;
+------+------------+-------------+--------+-------+--------+
| ID   | Nome       | Endereco    | Cidade | CEP   | Estado |
+------+------------+-------------+--------+-------+--------+
|    1 | Jose Maria | Rua 1, num 2 | Betim  | 12209 | MG     |
+------+------------+-------------+--------+-------+--------+
1 row in set (0.00 sec)
```

- Consultas com strings, usamos aspas duplas ou simples.
- Consultas com números, não usamos aspas.

Consultas com WHERE + operadores

Podemos usar os operadores de comparação:

- igual, =;
- maior, maior ou igual: >, >=;
- diferente, ! = ou <>;
- menor, menor ou igual: <, <=;

Podemos também usar as palavras chaves:

- BETWEEN
- LIKE
- IN

Podemos também usar funções matemáticas para filtrar dados. Uma função muito usada é a MOD() que retorna o resto da divisão inteira. Usamos MOD para, por exemplo, verificar se um número é par ou ímpar.

```
mysql> select mod(4,2);
+----------+
| mod(4,2) |
+----------+
|        0 |
+----------+
mysql> select mod(5,2);
+----------+
| mod(5,2) |
+----------+
|        1 |
+----------+
```

2.1 Exercício de Fixação

Faças as Consultas Pedidas

1. Selecione todos clientes com ID < 3. Mostre ID e Nome.
2. Selecione todos clientes com ID par. Mostre ID e Nome.
3. Selecione todos clientes cuja cidade começa com letra "B". Mostre ID, Nome e Cidade.
4. Selecione todos clientes com ID entre 3 e 5. Mostre ID e Nome.
5. Selecione todos os clientes dos estados do Nordeste. Mostre ID,Nome e Estado.
6. Selecione Nome, ID e Cidade dos clientes que se chamam João ou Joana.

Sugestão de Solução

```
mysql> #1) selecione todos clientes com ID < 3
mysql> select ID, Nome from Clientes where ID < 3;
+------+------------+
| ID   | Nome       |
+------+------------+
|    1 | Jose Maria |
|    2 | Maria Jose |
+------+------------+
2 rows in set (0.00 sec)

mysql> # 2) Selecione todos clientes com ID par. Mostre ID e Nome.
mysql> select ID,Nome from Clientes
    -> where mod(ID,2) = 0;
+------+-------------+
| ID   | Nome        |
+------+-------------+
|    2 | Maria Jose  |
|    4 | Joana Carla |
+------+-------------+
2 rows in set (0.00 sec)


mysql> # 3) Selecione todos clientes cuja cidade começa com letra "B". Mostre ID, Nome e C
mysql> select ID,Nome,Cidade from Clientes  where Cidade LIKE "B%";
+------+--------------+--------+
| ID   | Nome         | Cidade |
+------+--------------+--------+
|    1 | Jose Maria   | Betim  |
|    4 | Joana Carla  | BH     |
|    5 | João Carlos  | BH     |
+------+--------------+--------+
3 rows in set (0.00 sec)
```

```
mysql> # 4) Selecione todos clientes com ID entre 3 e 5. Mostre ID e Nome.
mysql> select ID,Nome,Cidade from Clientes  where ID BETWEEN 3 AND 5;
+------+--------------+---------+
| ID   | Nome         | Cidade  |
+------+--------------+---------+
|    3 | Ana Maria    | Aracaju |
|    4 | Joana Carla  | BH      |
|    5 | João Carlos  | BH      |
+------+--------------+---------+
3 rows in set (0.00 sec)
```

```
mysql> # 5) Selecione todos os clientes dos estados do Nordeste. Mostre ID,Nome e Estado.
mysql> select ID,Nome,Estado from Clientes  where Estado IN ('AL','SE');
+------+------------+--------+
| ID   | Nome       | Estado |
+------+------------+--------+
|    2 | Maria Jose | AL     |
|    3 | Ana Maria  | SE     |
+------+------------+--------+
2 rows in set (0.00 sec)
```

```
mysql> # 6) Selecione Nome, ID e Cidade dos clientes que se chamam João ou Joana.
mysql> select Nome,ID, Cidade from Clientes  where Nome LIKE "João%" or Nome LIKE "Joana%"
+--------------+------+--------+
| Nome         | ID   | Cidade |
+--------------+------+--------+
| Joana Carla  |    4 | BH     |
| João Carlos  |    5 | BH     |
+--------------+------+--------+
2 rows in set (0.00 sec)
```

- Repare que na questão 5 usamos a palavra-chave IN para verificar se o estado pesquisado está dentro do conjunto definido como ('AL','SE'). Essa é uma forma comum de avaliarmos se um valor está dentro de um conjunto de valores.
- Podemos usar os operadores lógicos AND, OR para pesquisarmos valores.

```
mysql> # 7) Selecione Nome, ID e Cidade dos clientes que se chamam João ou Joana.
mysql> select Nome,ID,Cidade from Clientes where Nome like "João%" or Nome Like "Joana%";
+--------------+------+--------+
| Nome         | ID   | Cidade |
+--------------+------+--------+
| Joana Carla  |    4 | BH     |
| João Carlos  |    5 | BH     |
+--------------+------+--------+
2 rows in set (0.00 sec)
```

A consulta a seguir usa o operador AND:

7. Mostre os clientes que tem nome que começam com letra "J"e moram em cidades que tem nome que começam com letra "B". Mostre o nome e a cidade.

```
mysql> # 8)  Mostre os clientes que tem nome que começam com letra "J"
e moram em cidades que tem nome que começam com letra "B".
Mostre o nome e a cidade.
mysql> select Nome,Cidade from Clientes where Nome like "J%" and Cidade Like "B%";
+--------------+--------+
| Nome         | Cidade |
+--------------+--------+
| Jose Maria   | Betim  |
| Joana Carla  | BH     |
| João Carlos  | BH     |
+--------------+--------+
3 rows in set (0.00 sec)
```

2.2 Tabela Produtos

Tabela 2: Tabela com Produtos

ID	ProductName	CategoriaID	Quantidade	Preco
1	HD	1	12	400
2	PlacaMae	1	15	1000
3	Memória RAM	2	14	500
4	Papel A4	3	33	40
5	Canetas	4	44	2
6	Marcador	4	22	3

Palavra-chave ORDER BY

Usamos ORDER BY para order em ordem crescente (ASC) ou decrescente (DESC).

8. Informe todos os produtos, ordenados pelo preço, em ordem descrescente.

```
mysql> # 8) Informe todos os produtos, ordenados pelo preço, em ordem descrescente.
mysql> select * from Produtos order by Preco desc;
+------+-------------+------------+-----------+-------+
| ID   | ProductName | CategoriaID | Quantidade | Preco |
+------+-------------+------------+-----------+-------+
|    2 | PlacaMae    |          1 |         15 |  1000 |
|    3 | Memória RAM |          2 |         14 |   500 |
|    1 | HD          |          1 |         12 |   400 |
|    4 | Papel A4    |          3 |         33 |    40 |
|    6 | Marcador    |          4 |         22 |     3 |
|    5 | Canetas     |          4 |         44 |     2 |
+------+-------------+------------+-----------+-------+
6 rows in set (0,00 sec)
```

9. Informe todos os produtos, ordenados pelo preço, em ordem crescente.

```
mysql> select * from Produtos order by Preco asc;
+------+-------------+------------+-----------+-------+
| ID   | ProductName | CategoriaID | Quantidade | Preco |
+------+-------------+------------+-----------+-------+
|    5 | Canetas     |          4 |         44 |     2 |
|    6 | Marcador    |          4 |         22 |     3 |
|    4 | Papel A4    |          3 |         33 |    40 |
|    1 | HD          |          1 |         12 |   400 |
|    3 | Memória RAM |          2 |         14 |   500 |
|    2 | PlacaMae    |          1 |         15 |  1000 |
+------+-------------+------------+-----------+-------+
6 rows in set (0,00 sec)
```

Podemos ordenar por mais de uma coluna, como mostra a próxima consulta.

10. Mostre todos os clientes, ordenados por estados e por nome.

```
mysql> # 10) Mostre todos os clientes, ordenados por estados e por nome.
mysql> select * from Clientes order by Estado, Nome;
+------+-------------+--------------+---------+-------+--------+
| ID   | Nome        | Endereco     | Cidade  | CEP   | Estado |
+------+-------------+--------------+---------+-------+--------+
|    2 | Maria Jose  | Rua 2, num 33 | Maceio  |  5021 | AL     |
|    4 | Joana Carla | Rua 6, 7     | BH      |   123 | MG     |
|    5 | João Carlos | Av. Flor     | BH      | 12345 | MG     |
|    1 | Jose Maria  | Rua 1, num 2 | Betim   | 12209 | MG     |
|    3 | Ana Maria   | Rua A, num 4 | Aracaju |  5023 | SE     |
+------+-------------+--------------+---------+-------+--------+
```

3 Revisitando Operadores Lógicos

Podemos usar combinações de Operadores Lógicos: AND, OR, NOT. Acompanhe as consultas a seguir.

11. Encontre os clientes de Minas Gerais que possuem o último nome "Maria"ou "Carla".

```
mysql> # 11) Encontre os clientes de Minas Gerais que possuem
o último nome "Maria" ou "Carla".
mysql> select * from Clientes
-> where Estado = 'MG' and (Nome like "%Maria" or Nome like "%Carla");
+------+-------------+--------------+--------+-------+--------+
| ID   | Nome        | Endereco     | Cidade | CEP   | Estado |
+------+-------------+--------------+--------+-------+--------+
|    1 | Jose Maria  | Rua 1, num 2 | Betim  | 12209 | MG     |
|    4 | Joana Carla | Rua 6, 7     | BH     |   123 | MG     |
+------+-------------+--------------+--------+-------+--------+
```

12. Encontre os Clientes que não são de Minas Gerais

```
mysql> # 12) Encontre os Clientes que não são de Minas Gerais
mysql> select * from Clientes where NOT Estado = 'MG';
+------+------------+---------------+---------+------+--------+
| ID   | Nome       | Endereco      | Cidade  | CEP  | Estado |
+------+------------+---------------+---------+------+--------+
|    2 | Maria Jose | Rua 2, num 33 | Maceio  | 5021 | AL     |
|    3 | Ana Maria  | Rua A, num 4  | Aracaju | 5023 | SE     |
+------+------------+---------------+---------+------+--------+
```

3.1 Sintaxe dos Operadores AND, OR, NOT

AND

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;
```

OR

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT

```
SELECT column1, column2, ...
FROM table_name
WHERE NOT condition;
```

4 Exercícios de Fixação

(adaptado de W3Schools: `www.w3schools.com/sql/sql_and.asp`)

Usando o Banco de Dados northwind2024, faça as consultas pedidas:

1. Mostre os clientes (Customers) que são da Alemanha e da cidade Berlin. Mostre os campos CustomerID,CompanyName,City,Country.

```
use northwind2024
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> WHERE Country = 'Germany' AND City = 'Berlin';
+------------+---------------------+--------+---------+
| CustomerID | CompanyName         | City   | Country |
+------------+---------------------+--------+---------+
| ALFKI      | Alfreds Futterkiste | Berlin | Germany |
+------------+---------------------+--------+---------+
```

2. Mostre os clientes que são de Berlin ou de Stuttgart.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> where City = "Berlin" OR City = "Stuttgart";
+------------+---------------------+-----------+---------+
| CustomerID | CompanyName         | City      | Country |
+------------+---------------------+-----------+---------+
| ALFKI      | Alfreds Futterkiste | Berlin    | Germany |
| WANDK      | Die Wandernde Kuh   | Stuttgart | Germany |
+------------+---------------------+-----------+---------+
```

3. Mostre os clientes que são da Espanha ou da Alemanha.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> WHERE Country = "Germany" OR Country = "Spain";
+------------+-------------------------------------+----------------+---------+
| CustomerID | CompanyName                         | City           | Country |
+------------+-------------------------------------+----------------+---------+
| ALFKI      | Alfreds Futterkiste                 | Berlin         | Germany |
| BLAUS      | Blauer See Delikatessen             | Mannheim       | Germany |
| BOLID      | Blido Comidas preparadas            | Madrid         | Spain   |
:
:
:
| ROMEY      | Romero y tomillo                    | Madrid         | Spain   |
| TOMSP      | Toms Spezialitten                   | Mnster         | Germany |
| WANDK      | Die Wandernde Kuh                   | Stuttgart      | Germany |
+------------+-------------------------------------+----------------+---------+
16 rows in set (0,00 sec)
```

4. Selecione os clientes que não são da Alemanha.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> where NOT Country = "Germany";
+------------+--------------------------------------+-----------------+-------------+
| CustomerID | CompanyName                          | City            | Country     |
+------------+--------------------------------------+-----------------+-------------+
| ANATR      | Ana Trujillo Emparedados y helados   | Mxico D.F.      | Mexico      |
:
:
:
| WELLI      | Wellington Importadora               | Resende         | Brazil      |
| WHITC      | White Clover Markets                 | Seattle         | USA         |
| WILMK      | Wilman Kala                          | Helsinki        | Finland     |
| WOLZA      | Wolski  Zajazd                       | Warszawa        | Poland      |
+------------+--------------------------------------+-----------------+-------------+
80 rows in set (0,00 sec)
```

5. Mostre os clientes que não são da Alemanha nem dos Estados Unidos.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> WHERE NOT Country = 'Germany' AND NOT Country = 'USA';
+------------+--------------------------------------+-----------------+-------------+
| CustomerID | CompanyName                          | City            | Country     |
+------------+--------------------------------------+-----------------+-------------+
| ANATR      | Ana Trujillo Emparedados y helados   | Mxico D.F.      | Mexico      |
| ANTON      | Antonio Moreno Taquera               | Mxico D.F.      | Mexico      |
:
:
| WELLI      | Wellington Importadora               | Resende         | Brazil      |
| WILMK      | Wilman Kala                          | Helsinki        | Finland     |
| WOLZA      | Wolski  Zajazd                       | Warszawa        | Poland      |
+------------+--------------------------------------+-----------------+-------------+
67 rows in set (0,00 sec)
```

5 Valores NULL

(adaptado de W3Schools: `https://www.w3schools.com/mysql/mysql_null_values.asp`)

Um campo com valor NULL é um campo sem valor.

Se um campo de uma tabela for opcional, é possível inserir um novo registro ou atualizar um registro sem adicionar um valor a este campo. Então, o campo será salvo com valor NULL.

Atenção: Um valor NULL é diferente de um valor zero ou de um campo que contém espaços. Um campo com valor NULL é aquele que foi deixado em branco durante a criação do registro!

5.1 Como testar valores NULL

Para testar se um valoe é ou não NULL, não podemos usar os operadores de comparação, como =, < ou <>.

Para testar se um valor é NULL temos que usar os operadores **IS NULL** e **IS NOT NULL**.

SINTAXE

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

ou, para NOT NULL:

```
SELECT column_names
FROM table_name
WHERE column_name IS NOT NULL;
```

Acompanhe as consultas para o banco northwind2024.

13. Mostre o ID, Nome e Endereço dos Clientes que possuem NULL para o campo "Address".

```
mysql> # 13) Mostre o Nome e Endereço dos Clientes que possuem NULL para o campo ''Ad
+------------+-------------+---------+
| CustomerID | ContactName | Address |
+------------+-------------+---------+
| Val2       | Val2        | NULL    |
| VALON      | Valon Hoti  | NULL    |
+------------+-------------+---------+
2 rows in set (0,00 sec)
```

14. Mostre o ID, Nome e Endereço dos Clientes que possuem valores para o campo "Address".

```
# 14) Mostre o ID, Nome e Endereço dos Clientes que possuem valores para o campo ''Ad
mysql> select CustomerID, ContactName, Address
    -> from Customers
    -> where Address IS NOT NULL;
+------------+------------------------+-------------------------------------------
| CustomerID | ContactName            | Address
+------------+------------------------+-------------------------------------------
| ALFKI      | Maria Anders           | Obere Str. 57
| ANATR      | Ana Trujillo           | Avda. de la Constitucin 2222
| ANTON      | Antonio Moreno         | Mataderos  2312
:
:
| WHITC      | Karl Jablonski         | 305 - 14th Ave. S. Suite 3B
| WILMK      | Matti Karttunen        | Keskuskatu 45
| WOLZA      | Zbyszek Piestrzeniewicz | ul. Filtrowa 68
+------------+------------------------+-------------------------------------------
91 rows in set (0,00 sec)
```

6 Funções MAX() e MIN()

Vamos continuar usando o banco de dados northwind2024.

Veja os 15 primeiros itens da tabela "Products", mostrando os campos ProductID, ProductName, UnitsInStock,UnitPrice.

```
mysql> select ProductID, ProductName, UnitsInStock,UnitPrice from Products limit 15;
+-----------+---------------------------------+--------------+-----------+
| ProductID | ProductName                     | UnitsInStock | UnitPrice |
+-----------+---------------------------------+--------------+-----------+
|         1 | Chai                            |           39 |   18.0000 |
|         2 | Chang                           |           17 |   19.0000 |
|         3 | Aniseed Syrup                   |           13 |   10.0000 |
|         4 | Chef Anton's Cajun Seasoning    |           53 |   22.0000 |
|         5 | Chef Anton's Gumbo Mix          |            0 |   21.3500 |
|         6 | Grandma's Boysenberry Spread    |          120 |   25.0000 |
|         7 | Uncle Bob's Organic Dried Pears |           15 |   30.0000 |
|         8 | Northwoods Cranberry Sauce      |            6 |   40.0000 |
|         9 | Mishi Kobe Niku                 |           29 |   97.0000 |
|        10 | Ikura                           |           31 |   31.0000 |
|        11 | Queso Cabrales                  |           22 |   21.0000 |
|        12 | Queso Manchego La Pastora       |           86 |   38.0000 |
|        13 | Konbu                           |           24 |    6.0000 |
|        14 | Tofu                            |           35 |   23.2500 |
|        15 | Genen Shouyu                    |           39 |   15.5000 |
+-----------+---------------------------------+--------------+-----------+
15 rows in set (0,00 sec)
```

15. Mostre o menor preço dentre todos os preços de todos produtos.

```
mysql> SELECT MIN(UnitPrice) AS "Menor Preço" FROM Products;
+--------------+
| Menor Preço  |
+--------------+
|       2.5000 |
+--------------+
1 row in set (0,00 sec)
```

16. Mostre o preço mais alto dentre todos os preços de todos produtos.

```
mysql> SELECT MAX(UnitPrice) AS "Maior Preço" FROM Products;
+--------------+
| Maior Preço  |
+--------------+
|     263.5000 |
+--------------+
1 row in set (0,01 sec)
```

7 Funções COUNT(), AVG(), SUM()

Seguem exemplos dessas três funções aplicadas a consultas no banco de dados northwind2024.

A função COUNT() retorna o número de linhas correspondentes a um critério especificado.

17. Informe quantos produtos há na tabela “Products”;

```
mysql> # 17) Informe quantos produtos há na tabela ‘‘Products’’;
mysql> SELECT COUNT(ProductID)
    -> FROM Products;
+------------------+
| COUNT(ProductID) |
+------------------+
|               77 |
+------------------+
1 row in set (0.00 sec)
```

18. Informe o valor médio dentre todos os preços informados na tabela “Products”;

```
mysql> # 18) Informe o valor médio dentre todos os preços informados na tabela ‘‘Prod
mysql> SELECT AVG(UnitPrice) FROM Products;
+-----------------+
| AVG(UnitPrice) |
+-----------------+
|    28.86636364 |
+-----------------+
1 row in set (0.00 sec)
```

19. Encontre total de itens que foram pedidos. Use a tabela OrderDetails, verificando a coluna ”Quantity”.

```
mysql> # 19) Encontre total de itens que foram pedidos. Use a tabela OrderDetails, ve
mysql> SELECT SUM(Quantity) FROM OrderDetails;
+---------------+
| SUM(Quantity) |
+---------------+
|         51317 |
+---------------+
1 row in set (0.00 sec)
```

8 Group By

Usamos a cláusula GROUP BY para agrupar registros em subgrupos baseados em colunas ou valores retornados por uma expressão.

Com o GROUP BY podemos agrupar os valores de uma coluna e também realizar cálculos sobre esses valores. Desta forma, ao realizarmos ma consulta, os valores encontrados nas linhas são agrupados e então uma função de agregação pode ser aplicada sobre esses grupos.

A instrução GROUP BY é frequentemente usada com funções agregadoras (COUNT(), MAX(), MIN(), SUM(), AVG()) para agrupar o conjunto de resultados por uma ou mais colunas.

A sintaxe básica para uso do **group by** é:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

8.1 Exemplos com GROUP BY

20. Encontre o número total de clientes de cada país.

```
mysql> SELECT COUNT(CustomerID), Country
    -> FROM Customers
    -> GROUP BY Country;
+-------------------+-------------+
| COUNT(CustomerID) | Country     |
+-------------------+-------------+
|                11 | Germany     |
|                 5 | Mexico      |
|                 7 | UK          |
:
:
|                 2 | NULL        |
|                 2 | Finland     |
|                 1 | Poland      |
+-------------------+-------------+
22 rows in set (0,00 sec)
```

21. Podemos ordenar a consulta acima:

```
mysql> SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country
    -> order by Country ASC;
+-------------------+-------------+
| COUNT(CustomerID) | Country     |
+-------------------+-------------+
|                 2 | NULL        |
|                 3 | Argentina   |
|                 2 | Austria     |
|                 2 | Belgium     |
:
:
|                 7 | UK          |
|                13 | USA         |
|                 4 | Venezuela   |
+-------------------+-------------+
22 rows in set (0,00 sec)
```

22. Podemos ordenar em relação ao número de clientes:

```
mysql> SELECT COUNT(CustomerID) as "Número Clientes", Country FROM Customers GROUP BY
+------------------+-------------+
| Número Clientes  | Country     |
+------------------+-------------+
|                1 | Poland      |
|                1 | Norway      |
|                1 | Ireland     |
|                2 | Finland     |
|                2 | Sweden      |
|                2 | NULL        |
|                2 | Denmark     |
|                2 | Belgium     |
|                2 | Switzerland |
|                2 | Austria     |
|                2 | Portugal    |
|                3 | Italy       |
|                3 | Argentina   |
|                3 | Canada      |
|                4 | Venezuela   |
|                5 | Spain       |
|                5 | Mexico      |
|                7 | UK          |
|                9 | Brazil      |
|               11 | Germany     |
|               11 | France      |
|               13 | USA         |
+------------------+-------------+
22 rows in set (0,00 sec)
```

8.2 Group By sem Funções Agregadoras

Não é comum usarmos o group by sem ter uma agregação (como o cálculo da soma, da média, ou a busca de valores máximos ou mínimos). Geralmente agrupamos algo quando queremos ver valores totais ou médias do grupo, por exemplo.

Uma situação de consulta com group by sem função de agregação é para buscarmos valores distintos. Veja a consulta a seguir, realizada no banco de dados northwind2024.

```
mysql> select FirstName, LastName, Address, City, Country from Employees;
+-----------+-----------+--------------------------------+----------+---------+
| FirstName | LastName  | Address                        | City     | Country |
+-----------+-----------+--------------------------------+----------+---------+
| Nancy     | Davolio   | 507 - 20th Ave. E.Apt. 2A      | Seattle  | USA     |
| Andrew    | Fuller    | 908 W. Capital Way             | Tacoma   | USA     |
| Janet     | Leverling | 722 Moss Bay Blvd.             | Kirkland | USA     |
| Margaret  | Peacock   | 4110 Old Redmond Rd.           | Redmond  | USA     |
| Steven    | Buchanan  | 14 Garrett Hill                | London   | UK      |
| Michael   | Suyama    | Coventry House
Miner Rd.       | London   | UK      |
| Robert    | King      | Edgeham Hollow
Winchester Way | London   | UK      |
| Laura     | Callahan  | 4726 - 11th Ave. N.E.          | Seattle  | USA     |
| Anne      | Dodsworth | 7 Houndstooth Rd.              | London   | UK      |
+-----------+-----------+--------------------------------+----------+---------+
9 rows in set (0,00 sec)
```

Se quisermos agrupar por país nossa consulta podemos fazer:

```
mysql> select Country from Employees group by Country;
+---------+
| Country |
+---------+
| USA     |
| UK      |
+---------+
2 rows in set (0,00 sec)
```

O que é a mesma coisa de fazermos:

```
mysql> select DISTINCT Country from Employees;
+---------+
| Country |
+---------+
| USA     |
| UK      |
+---------+
2 rows in set (0,00 sec)
```

Agora, se tentarmos agrupar por país e mostrarmos por exemplo o primeiro nome de cada empregado, geraríamos um erro:

```
mysql> select FirstName,Country from Employees group by Country;
ERROR 1055 (42000): Expression #1 of SELECT list is not in GROUP BY clause
and contains nonaggregated column 'northwind2024.Employees.FirstName'
which is not functionally dependent on columns in GROUP BY clause;
this is incompatible with sql_mode=only_full_group_by
```

8.3 Exercícios de Fixação

1. Mostre os IDs das regiões e o total de territórios de cada uma.
2. Mostre as diferentes categorias de produtos e o valor total dos preços de cada uma.
3. Mostre quantos registros possui a tabela OrderDetails;
4. Mostre quantos pedidos foram feitos, verificando quantos pedidos estão registrados na tabela OrderDetails. Use o alias "Total de Pedidos" para o resultado.
5. Mostre quantos pedidos foram feitos, consultando a tabela Orders.
6. Liste os pedidos feitos, informando o ID do pedido (OrderID), o preço de cada unidade e a quantidade pedida. Use a tabela OrderDetails.
7. Repita a consulta acima, limitando a consulta aos 10 primeiros registros.
8. Repita a consulta dada no item 6, limitando aos 10 últimos registros.
9. Informe o número do pedido, o valor unitário, quantidade e o preço total de cada pedido. O preço total é dado por (preço unitário) $\times$ (quantidade).
10. Refaça a consulta anterior, agora mostrando os valores totais de Preço Unitário, de Quantidade e do Preço Total para cada OrderID (ou seja agrupado por número de pedido).
11. Refaça a consulta anterior, agora mostrando o 10 maiores "Preços Totais".
12. Faça uma consulta mostrando o ID do pedido (OrderID) e o preço total do pedido. Use o apelido "Preço Total". Mostre os 10 primeiros pedidos com maiores valores totais

9 Soluções

Sugestões de soluções para os exercícios da seção 8.3.

```
# Exercício de Fixação
# 1) Mostre os IDs das regiões e o total de territórios de cada uma.
select RegionID,count(TerritoryID) from Territories group by RegionID;
+----------+--------------------+
| RegionID | count(TerritoryID) |
+----------+--------------------+
|        1 |                 19 |
|        2 |                 15 |
|        3 |                 11 |
|        4 |                  8 |
+----------+--------------------+
4 rows in set (0,00 sec)

# 2) Mostre as diferentes categorias de produtos e o valor total dos preços de cada uma.
select CategoryID,sum(UnitPrice) from Products group by CategoryID;
+------------+----------------+
| CategoryID | sum(UnitPrice) |
+------------+----------------+
|          1 |       455.7500 |
|          2 |       276.7500 |
|          3 |       327.0800 |
|          4 |       287.3000 |
|          5 |       141.7500 |
|          6 |       324.0400 |
|          7 |       161.8500 |
|          8 |       248.1900 |
+------------+----------------+
8 rows in set (0,00 sec)

#  3) Mostre quantos registros possui a tabela OrderDetails;
mysql> select count(*) from OrderDetails;
+----------+
| count(*) |
+----------+
|     2155 |
+----------+
1 row in set (0,00 sec)

# 4. Mostre quantos pedidos foram feitos, verificando quantos pedidos estão registrados na
OrderDetails. Use o alias ''Total de Pedidos'' para o resultado.

mysql> select count(distinct OrderID) as "Total de Pedidos" from OrderDetails;
+------------------+
| Total de Pedidos |
+------------------+
|              830 |
+------------------+
1 row in set (0,00 sec)
```

```
# 5) Mostre quantos pedidos foram feitos, consultando a tabela Orders.
mysql> select count(*) 'Total de Pedidos' from Orders;
+------------------+
| Total de Pedidos |
+------------------+
|              830 |
+------------------+
1 row in set (0,00 sec)
ou
mysql> select count(Distinct OrderID)  'Total de Pedidos'
    -> from Orders;
+------------------+
| Total de Pedidos |
+------------------+
|              830 |
+------------------+

# 6) Liste os pedidos feitos, informando o ID do pedido (OrderID),
o preço de cada unidade e a quantidade pedida. Use a tabela OrderDetails.

mysql> select OrderID, UnitPrice, Quantity  from OrderDetails;
+---------+-----------+----------+
| OrderID | UnitPrice | Quantity |
+---------+-----------+----------+
|   10248 |   14.0000 |       12 |
|   10248 |    9.8000 |       10 |
|   10248 |   34.8000 |        5 |
|   10249 |   18.6000 |        9 |
|   10249 |   42.4000 |       40 |
|   10250 |    7.7000 |       10 |
|   10250 |   42.4000 |       35 |
|   10250 |   16.8000 |       15 |
|   10251 |   16.8000 |        6 |
|   10251 |   15.6000 |       15 |
:
:
:
|   11077 |   33.2500 |        2 |
|   11077 |   17.0000 |        1 |
|   11077 |   15.0000 |        2 |
|   11077 |    7.7500 |        4 |
|   11077 |   13.0000 |        2 |
+---------+-----------+----------+
2155 rows in set (0,00 sec)
```

```
# 7)  Repita a consulta acima, limitando a consulta aos 10 primeiros registros.
mysql> select OrderID, UnitPrice, Quantity  from OrderDetails limit 10;
+---------+-----------+----------+
| OrderID | UnitPrice | Quantity |
+---------+-----------+----------+
|   10248 |   14.0000 |       12 |
|   10248 |    9.8000 |       10 |
|   10248 |   34.8000 |        5 |
|   10249 |   18.6000 |        9 |
|   10249 |   42.4000 |       40 |
|   10250 |    7.7000 |       10 |
|   10250 |   42.4000 |       35 |
|   10250 |   16.8000 |       15 |
|   10251 |   16.8000 |        6 |
|   10251 |   15.6000 |       15 |
+---------+-----------+----------+
10 rows in set (0,00 sec)


# 8) Repita a  consulta dada no item 6, limitando aos 10 últimos registros.

mysql> select OrderID, UnitPrice, Quantity  from OrderDetails limit 2145,10;
+---------+-----------+----------+
| OrderID | UnitPrice | Quantity |
+---------+-----------+----------+
|   11077 |    9.6500 |        3 |
|   11077 |   12.0000 |        3 |
|   11077 |    7.0000 |        2 |
|   11077 |   24.0000 |        2 |
|   11077 |   34.0000 |        2 |
|   11077 |   33.2500 |        2 |
|   11077 |   17.0000 |        1 |
|   11077 |   15.0000 |        2 |
|   11077 |    7.7500 |        4 |
|   11077 |   13.0000 |        2 |
+---------+-----------+----------+
10 rows in set (0,01 sec)

# repare o uso do limit. Nesse caso fizemos a consulta a partir do registro 2145, e limita
```

9) Informe o número do pedido, o valor unitário, quantidade e o preço total de cada pedido

```
select OrderID, UnitPrice, Quantity, UnitPrice*Quantity as PrecoTotal from
OrderDetails;
+---------+-----------+----------+------------+
| OrderID | UnitPrice | Quantity | PrecoTotal |
+---------+-----------+----------+------------+
|   10248 |   14.0000 |       12 |   168.0000 |
|   10248 |    9.8000 |       10 |    98.0000 |
|   10248 |   34.8000 |        5 |   174.0000 |
|   10249 |   18.6000 |        9 |   167.4000 |
|   10249 |   42.4000 |       40 |  1696.0000 |
|   10250 |    7.7000 |       10 |    77.0000 |
|   10250 |   42.4000 |       35 |  1484.0000 |
|   10250 |   16.8000 |       15 |   252.0000 |
|   10251 |   16.8000 |        6 |   100.8000 |
|   10251 |   15.6000 |       15 |   234.0000 |

:
:
:
|  11077  |   12.0000 |        3 |    36.0000 |
|   11077 |    7.0000 |        2 |    14.0000 |
|   11077 |   24.0000 |        2 |    48.0000 |
|   11077 |   34.0000 |        2 |    68.0000 |
|   11077 |   33.2500 |        2 |    66.5000 |
|   11077 |   17.0000 |        1 |    17.0000 |
|   11077 |   15.0000 |        2 |    30.0000 |
|   11077 |    7.7500 |        4 |    31.0000 |
|   11077 |   13.0000 |        2 |    26.0000 |
+---------+-----------+----------+------------+
2155 rows in set (0,01 sec)
```

```
# 10) Refaça a consulta anterior, agora mostrando os valores totais de Preço Unitário,
 de Quantidade e do Preço Total para cada  OrderID (ou seja agrupado por número de pedido)

mysql> select OrderID, Sum(UnitPrice), Sum(Quantity), Sum(UnitPrice*Quantity) as PrecoTota
 from OrderDetails group by OrderID;
+---------+----------------+---------------+------------+
| OrderID | Sum(UnitPrice) | Sum(Quantity) | PrecoTotal |
+---------+----------------+---------------+------------+
|   10248 |        58.6000 |            27 |   440.0000 |
|   10249 |        61.0000 |            49 |  1863.4000 |
|   10250 |        66.9000 |            60 |  1813.0000 |
|   10251 |        49.2000 |            41 |   670.8000 |
|   10252 |        94.0000 |           105 |  3730.0000 |
|   10253 |        40.4000 |           102 |  1444.8000 |
|   10254 |        30.8000 |            57 |   625.2000 |
|   10255 |        88.3000 |           110 |  2490.5000 |
|   10256 |        36.6000 |            27 |   517.8000 |
|   10257 |        59.9000 |            46 |  1119.9000 |
```

```
:
:
|   11071 |        36.0000 |            25 |   510.0000 |
|   11072 |        78.1500 |           200 |  5218.0000 |
|   11073 |        25.5000 |            30 |   300.0000 |
|   11074 |        17.4500 |            14 |   244.3000 |
|   11075 |        49.0000 |            42 |   586.0000 |
|   11076 |        57.4500 |            50 |  1057.0000 |
|   11077 |       574.3500 |            72 |  1374.6000 |
+---------+----------------+---------------+------------+
830 rows in set (0,00 sec)
```

11) Refaça a consulta anterior, agora mostrando o 10 maiores ‘‘Preços Totais’’.

```
mysql> select OrderID, Sum(UnitPrice), Sum(Quantity),
Sum(UnitPrice*Quantity) as PrecoTotal from OrderDetails
group by OrderID order by PrecoTotal desc limit 10;
+---------+----------------+---------------+------------+
| OrderID | Sum(UnitPrice) | Sum(Quantity) | PrecoTotal |
+---------+----------------+---------------+------------+
|   10865 |       281.5000 |           140 | 17250.0000 |
|   11030 |       219.1400 |           330 | 16321.9000 |
|   10981 |       263.5000 |            60 | 15810.0000 |
|   10372 |       330.6000 |           164 | 12281.2000 |
|   10424 |       235.2000 |           139 | 11493.2000 |
|   10817 |       362.4300 |           155 | 11490.7000 |
|   10889 |       284.5000 |            80 | 11380.0000 |
|   10417 |       240.8000 |           123 | 11283.2000 |
|   10897 |       149.6800 |           116 | 10835.2400 |
|   10353 |       227.6000 |            62 | 10741.6000 |
+---------+----------------+---------------+------------+
10 rows in set (0,00 sec)
```

12) Faça uma consulta mostrando o ID do pedido (OrderID) e o preço total do pedido. Use

```
mysql>  select OrderID, Sum(UnitPrice*Quantity) as "Preço Total" from OrderDetails group b
+---------+--------------+
| OrderID | Preço Total  |
+---------+--------------+
|   10865 |   17250.0000 |
|   11030 |   16321.9000 |
|   10981 |   15810.0000 |
|   10372 |   12281.2000 |
|   10424 |   11493.2000 |
|   10817 |   11490.7000 |
|   10889 |   11380.0000 |
|   10417 |   11283.2000 |
|   10897 |   10835.2400 |
|   10353 |   10741.6000 |
+---------+--------------+
10 rows in set (0,00 sec)
```

9.1 Exercício de Fixação

Nas consultas pedidas a seguir utilizamos a tabela 'Suppliers'.

1. Informe o ID, o nome da empresa, a cidade, a região e o país de todos os fornecedores salvos no banco de dados.

```
select SupplierID, CompanyName, City,Region,Country from Suppliers;
```

2. Liste todos os países e o número de fornecedores que cada um páis possui. Ordene por ordem descrescente do número de fornecedores.

```
select count(SupplierID),Country from Suppliers
    -> group by Country
    -> order by Count(SupplierID) desc;
```

3. Liste o ID, o nome da empresa, a cidade, a região e o país de todos os fornecedores salvos no banco de dados cuja região informada não está vazia (NULL).

```
select SupplierID, CompanyName, City,Region,Country from Suppliers
    -> where Region is not null;
```

4. Liste o ID, o nome da empresa, a cidade, a região e o país dos países nórdicos.

```
 select SupplierID, CompanyName, City,Region,Country from Suppliers
  where Country in ('Finland','Norway','Denmark','Sweden');
```

10 Usando a Cláusula *Having*

Quando usamos funções de agregação não podemos usar WHERE para testar condições. Nesses casos temos que usar a palavra-chave *HAVING*.

A sintaxe para o uso da cláusula *having* é:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Usando a Cláusula HAVING no banco de dados northwind2024

23. Informe o número de clientes de cada país.

```
mysql> select count(CustomerID), Country
    -> from Customers
    -> GROUP BY Country;
+------------------+-------------+
| count(CustomerID) | Country     |
+------------------+-------------+
|                11 | Germany     |
|                 5 | Mexico      |
|                 7 | UK          |
|                 2 | Sweden      |
|                11 | France      |
|                 5 | Spain       |
|                 3 | Canada      |
|                 3 | Argentina   |
|                 2 | Switzerland |
|                 9 | Brazil      |
|                 2 | Austria     |
|                 3 | Italy       |
|                 2 | Portugal    |
|                13 | USA         |
|                 4 | Venezuela   |
|                 1 | Ireland     |
|                 2 | Belgium     |
|                 1 | Norway      |
|                 2 | Denmark     |
|                 2 | NULL        |
|                 2 | Finland     |
|                 1 | Poland      |
+------------------+-------------+
22 rows in set (0,01 sec)
```

24. Agora informe o número de clientes de cada páis considerando apenas os países que possuem mais de 5 clientes.

```
mysql> SELECT COUNT(CustomerID), Country
    -> FROM Customers
    -> GROUP BY Country
    -> HAVING COUNT(CustomerID) > 5;
+------------------+---------+
| COUNT(CustomerID) | Country |
+------------------+---------+
|                11 | Germany |
|                 7 | UK      |
|                11 | France  |
|                 9 | Brazil  |
|                13 | USA     |
+------------------+---------+
```

25. Repita a consulta anterior, agora ordenando pelo número de Clientes decrescente.

```
mysql> SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country HAVING COUNT
    -> order by Count(CUstomerID) desc;
+-------------------+---------+
| COUNT(CustomerID) | Country |
+-------------------+---------+
|                13 | USA     |
|                11 | Germany |
|                11 | France  |
|                 9 | Brazil  |
|                 7 | UK      |
+-------------------+---------+
5 rows in set (0,00 sec)
```

26. Selecione os IDs e os Preços dos produtos que possuem $100 <$ Preço < 1000

```
mysql> select distinct ProductID, UnitPrice from OrderDetails
 where UnitPrice > 100 and UnitPrice <1000;
+-----------+-----------+
| ProductID | UnitPrice |
+-----------+-----------+
|        38 |  210.8000 |
|        29 |  123.7900 |
|        38 |  263.5000 |
+-----------+-----------+
3 rows in set (0,00 sec)
```

Pergunta: por que usar *DISTINCT* na consulta acima?

27. Repita a consulta anterior agora usando a palavra-chave BETWEEN:

```
mysql> select distinct ProductID, UnitPrice from OrderDetails
where UnitPrice between 100 and 1000;
+-----------+-----------+
| ProductID | UnitPrice |
+-----------+-----------+
|        38 |  210.8000 |
|        29 |  123.7900 |
|        38 |  263.5000 |
+-----------+-----------+
3 rows in set (0,00 sec)
```

28. Informe o ID do pedido, o preço total por unidade, a quantidade total pedida o valor total do pedido (quantidade total $\times$ preço unitário). Ordene pelo valor total maior, decrescente. Mostre apenas os pedidos que possuem o valor total acima de 30000 (trinta mil).

```
mysql>  select OrderID, sum(UnitPrice) as "Preço Unitário Total",
     sum(Quantity) as "Quantidade Total do Pedido",
     sum(UnitPrice)*sum(Quantity) as "Valor Total Pedido"
     from OrderDetails group by OrderID
     having sum(UnitPrice)*sum(Quantity) > 30000
     order by sum(UnitPrice)*sum(Quantity) desc;
+---------+-----------------------+---------------------------+--------------------
| OrderID | Preço Unitário Total  | Quantidade Total do Pedido | Valor Total Pedido
+---------+-----------------------+---------------------------+--------------------
|   11030 |              219.1400 |                       330 |         72316.2000
|   10817 |              362.4300 |                       155 |         56176.6500
|   10515 |              194.8500 |                       286 |         55727.1000
|   10372 |              330.6000 |                       164 |         54218.4000
|   10540 |              317.2300 |                       165 |         52342.9500
|   10360 |              368.1000 |                       138 |         50797.8000
|   10691 |              256.5400 |                       182 |         46690.2800
|   10479 |              307.6000 |                       148 |         45524.8000
|   10514 |              193.4000 |                       233 |         45062.2000
|   11077 |              574.3500 |                        72 |         41353.2000
|   10865 |              281.5000 |                       140 |         39410.0000
|   11021 |              219.0300 |                       168 |         36797.0400
|   10979 |              172.8000 |                       207 |         35769.6000
|   10847 |              118.2000 |                       288 |         34041.6000
|   10424 |              235.2000 |                       139 |         32692.8000
|   10612 |              122.0000 |                       263 |         32086.0000
|   10440 |              150.9000 |                       208 |         31387.2000
|   11032 |              337.5000 |                        90 |         30375.0000
|   10351 |              250.8000 |                       120 |         30096.0000
+---------+-----------------------+---------------------------+--------------------
19 rows in set (0,00 sec)
```

29. Liste todos os empregados (Nome e Sobrenome), seus países e salários.

```
mysql> select FirstName, LastName, Country, Salary  from Employees;
+-----------+-----------+---------+---------+
| FirstName | LastName  | Country | Salary  |
+-----------+-----------+---------+---------+
| Nancy     | Davolio   | USA     | 2954.55 |
| Andrew    | Fuller    | USA     | 2254.49 |
| Janet     | Leverling | USA     | 3119.15 |
| Margaret  | Peacock   | USA     | 1861.08 |
| Steven    | Buchanan  | UK      | 1744.21 |
| Michael   | Suyama    | UK      | 2004.07 |
| Robert    | King      | UK      | 1991.55 |
| Laura     | Callahan  | USA     |  2100.5 |
| Anne      | Dodsworth | UK      | 2333.33 |
+-----------+-----------+---------+---------+
9 rows in set (0,00 sec)
```

30. Liste a média dos salários por país.

```
mysql> select avg(Salary), Country  from Employees
group by Country order by sum(Salary) desc;
+-------------------+---------+
| avg(Salary)       | Country |
+-------------------+---------+
| 2457.9539794921875 | USA     |
| 2018.2900085449219 | UK      |
+-------------------+---------+
2 rows in set (0,00 sec)
```

31. Liste os empregados que possuem salários maiores que 2100.

```
mysql> select FirstName, LastName, Country, Salary  from Employees
    -> where Salary > 2100;
+-----------+-----------+---------+---------+
| FirstName | LastName  | Country | Salary  |
+-----------+-----------+---------+---------+
| Nancy     | Davolio   | USA     | 2954.55 |
| Andrew    | Fuller    | USA     | 2254.49 |
| Janet     | Leverling | USA     | 3119.15 |
| Laura     | Callahan  | USA     |  2100.5 |
| Anne      | Dodsworth | UK      | 2333.33 |
+-----------+-----------+---------+---------+
5 rows in set (0,00 sec)
```

32. Liste o(s) país(es) que possui(em) o total de gasto(s) por salário maior que 10000.

```
mysql> select sum(Salary), Country  from Employees group by Country
    -> having sum(Salary) > 10000
    -> order by sum(Salary) desc;
+-------------------+---------+
| sum(Salary)       | Country |
+-------------------+---------+
| 12289.769897460938 | USA     |
+-------------------+---------+
1 row in set (0,00 sec)
```